CD Lab 6

Name - Rithvik Rajesh Matta
Class - 6H
SRN - PES2UG23CS485

Parser.y

Open | parser.y ~/sem6/CD/PE-6 | Save

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern int yylex();
extern FILE *yyin;
void yyerror(const char *s);

FILE *outfile;

/* Temporary counter */
int temp_count = 1;

/* Function to generate new temporaries */
char* newTemp() {
    char *temp = (char*)malloc(10);
    sprintf(temp, "t%d", temp_count++);
    return temp;
}
%}

/* Make these visible in parser.tab.h */
%code requires {
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
}

/* Semantic values */
%union {
    char* str;
}

%token <str> ID NUM
%type <str> expr

%left '+' '-'
%left '*' '/'

%%

input:
      /* empty */
    | input line
;

line:
      ID '=' expr '\n' {
            fprintf(outfile, "(=, %s, -, %s)\n\n", $3, $1);
```

Yacc | Tab Width: 8 | Ln 110, Col 2 | INS

Text Editor

Open | parser.y ~/sem6/CD/PE-6 | Save

```
line:
      ID '=' expr '\n' {
            fprintf(outfile, "(=, %s, -, %s)\n\n", $3, $1);
            temp_count = 1;   /* ✅ RESET TEMP COUNTER PER LINE */
      }
    | '\n'
;

expr:
      expr '+' expr {
            char *t = newTemp();
            fprintf(outfile, "(+, %s, %s, %s)\n", $1, $3, t);
            $$ = t;
      }
    | expr '-' expr {
            char *t = newTemp();
            fprintf(outfile, "(-, %s, %s, %s)\n", $1, $3, t);
            $$ = t;
      }
    | expr '*' expr {
            char *t = newTemp();
            fprintf(outfile, "(*, %s, %s, %s)\n", $1, $3, t);
            $$ = t;
      }
    | expr '/' expr {
            char *t = newTemp();
            fprintf(outfile, "(/, %s, %s, %s)\n", $1, $3, t);
            $$ = t;
      }
    | '(' expr ')' {
            $$ = $2;
      }
    | ID {
            $$ = strdup($1);
      }
    | NUM {
            $$ = strdup($1);
      }
;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    yyin = fopen("input.txt", "r");
    outfile = fopen("output.txt", "w");
```

Yacc | Tab Width: 8 | Ln 110, Col 2 | INS

```c
void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main() {
    yyin = fopen("input.txt", "r");
    outfile = fopen("output.txt", "w");

    if (!yyin || !outfile) {
        printf("Error opening input/output file\n");
        return 1;
    }

    yyparse();

    fclose(yyin);
    fclose(outfile);

    printf("IC generated successfully in output.txt\n");
    return 0;
}
```

Yacc Tab Width: 8 Ln 110, Col 2 INS

Lexer.l

Open

lexer.l
~/sem6/CD/PE-6

Save

```
%option noyywrap

%{
#include "parser.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%%

[a-zA-Z][a-zA-Z0-9]* {
    yylval.str = strdup(yytext);
    return ID;
}

[0-9]+ {
    yylval.str = strdup(yytext);
    return NUM;
}

[ \t]+    ;

\n        return '\n';

"="       return '=';
"+"       return '+';
"-"       return '-';
"*"       return '*';
"/"       return '/';
"("       return '(';
")"       return ')';

. {
    printf("Invalid char: %s\n", yytext);
}

%%
```

[run.sh]

run.sh
~/sem6/CD/PE-6

```bash
#!/bin/bash

echo "Cleaning..."
rm -f parser.tab.* lex.yy.c ic_generator output.txt

echo "Bison..."
bison -d parser.y

echo "Flex..."
flex lexer.l

echo "Compiling..."
gcc parser.tab.c lex.yy.c -o ic_generator

echo "Running..."
./ic_generator

echo "Done! Check output.txt"
```

Input.txt

input.txt
~/sem6/CD/PE-6

```
a=b+c*d
x=(a+b)*(c-d)
y=a+b+c
```

Output.txt

Open

output.txt
~/sem6/CD/PE-6

Save

```
(*, c, d, t1)
(+, b, t1, t2)
(=, t2, -, a)

(+, a, b, t1)
(-, c, d, t2)
(*, t1, t2, t3)
(=, t3, -, x)

(+, a, b, t1)
(+, t1, c, t2)
(=, t2, -, y)
```